

LeetCode Deep Dive

Assignment Problems: When Bitmask DP Beats (and When It Secretly Is) Flow
Week 10 Supplement – TOAE209/TOAH209: Design and Analysis of Algorithms

Foreign Trade University

Outline

Why these two problems

- This week's official LeetCode list is Hard-heavy for a reason: LEETCODE.md notes “pure max-flow problems are uncommon on LeetCode.” Instead, the list is full of **assignment**-flavored problems – bipartite point-to-point matching with a cost to minimize – which is exactly what the Max-Flow Min-Cut Theorem was built to solve.
- We pick one **Medium** (*Campus Bikes II*) and one **Hard** (*Minimum Cost to Connect Two Groups of Points*) – both look like “bipartite cost-minimization,” but only *one* of them is actually a flow problem in disguise.
- That distinction – when the beautiful flow/cut theorem applies, and when it quietly does not – is the real lesson of this deck.

Problem statement

LeetCode 1066 – Campus Bikes II (Medium)

Given n workers and m bikes ($n \leq m$) as 2D coordinates, assign each worker a **distinct** bike so as to minimize the **sum** of Manhattan distances between each worker and their assigned bike. Return that minimum sum.

- Constraints (LeetCode): $n, m \leq 10$, coordinates in $[0, 100)$.
- This is the textbook **assignment problem**: a perfect matching in a weighted bipartite graph, minimizing total edge weight – exactly the kind of problem Kleinberg & Tardos Ch. 7 reduces to min-cost *flow*.

Worked example

workers = $[[0,0],[2,1]]$, bikes = $[[1,2],[3,3]]$

	bike 0 = (1, 2)	bike 1 = (3, 3)
worker 0 = (0, 0)	$ 0-1 + 0-2 = 3$	$ 0-3 + 0-3 = 6$
worker 1 = (2, 1)	$ 2-1 + 1-2 = 2$	$ 2-3 + 1-3 = 3$

- Assign $w_0 \rightarrow b_0$ (3) and $w_1 \rightarrow b_1$ (3): total = 6.
- Assign $w_0 \rightarrow b_1$ (6) and $w_1 \rightarrow b_0$ (2): total = 8 – worse.
- **Answer:** 6.

First instinct: try every assignment

Most direct reading of the problem

“Assign each worker a distinct bike” $\Rightarrow$ enumerate every way to pick n distinct bikes out of m and assign them to the n workers, keep the cheapest.

- Number of assignments: $\frac{m!}{(m-n)!}$ (a permutation, not just a combination – *which* worker gets *which* bike matters).
- At the LeetCode-legal maximum $n = m = 10$: $10! = 3,628,800$ – survives only because the constraints are tiny. Relax n, m to 20: $20!/10! \approx 6.7 \times 10^{13}$ – gone.

Why this is bad

- Brute-force assignment is factorial in $\min(n, m)$ – it explodes long before ordinary polynomial algorithms would even notice the input size.
- It also re-derives, from scratch, a fact that decades of combinatorial optimization already solved in polynomial time: minimum-cost *perfect* bipartite matching is exactly an instance of min-cost flow (source $\rightarrow$ every worker $\rightarrow$ every bike $\rightarrow$ sink, unit capacities, edge cost = distance).

The insight to notice

Whenever a problem says “assign distinct items on the left to distinct items on the right, minimizing total cost,” that is the assignment problem – a *special case* of min-cost flow with all capacities equal to 1, solvable in polynomial time. You never need to enumerate permutations.

The structural insight, made practical

- In principle: build the flow network above and run a min-cost-flow algorithm – polynomial time, and (per this week's supplement on advances) as fast as $m^{1+o(1)}$ with 2022–2025's algorithms.
- In practice, at $n, m \leq 10$: the state needed to describe “which bikes are already used” fits in a single machine word – a **bitmask** over $m \leq 10$ bikes. That turns the assignment problem into a tiny dynamic program, simpler to code than a flow solver and just as optimal at this size.
- $dp[i][mask]$ = minimum cost to have assigned the first i workers, using exactly the bikes in $mask$.

Optimal algorithm: pseudocode

```
1: function CAMPUSBIKESII(workers, bikes)
2:    $n \leftarrow |\text{workers}|$ ,  $m \leftarrow |\text{bikes}|$ 
3:    $dp[0][0] \leftarrow 0$ ; all other  $dp[0][\cdot] \leftarrow \infty$ 
4:   for  $i \leftarrow 0$  to  $n - 1$  do
5:     for each mask with  $dp[i][\text{mask}] < \infty$  do
6:       for  $j \leftarrow 0$  to  $m - 1$  with bit  $j$  not in mask do
7:          $\text{mask}' \leftarrow \text{mask} \cup \{j\}$ 
8:          $dp[i+1][\text{mask}'] \leftarrow \min(dp[i+1][\text{mask}'], dp[i][\text{mask}] + \text{dist}(w_i, b_j))$ 
9:       end for
10:    end for
11:  end for
12:  return  $\min_{\text{mask}: |\text{mask}|=n} dp[n][\text{mask}]$ 
13: end function
```

Cost: $O(n \cdot 2^m \cdot m)$ – at $n, m \leq 10$, about $10 \times 1024 \times 10 \approx 10^5$ operations.

Trace on the worked example

Bits: bit 0 = bike0, bit 1 = bike1. $dp[0][00_2] = 0$.

step	transition	new state	value
1	$w_0 \rightarrow b_0$ (cost 3)	$dp[1][01_2]$	$0 + 3 = 3$
2	$w_0 \rightarrow b_1$ (cost 6)	$dp[1][10_2]$	$0 + 6 = 6$
3	from $dp[1][01_2]=3$: $w_1 \rightarrow b_1$ (cost 3)	$dp[2][11_2]$	$3 + 3 = 6$
4	from $dp[1][10_2]=6$: $w_1 \rightarrow b_0$ (cost 2)	$dp[2][11_2]$	$\min(6, 6+2) = 6$

Answer: $dp[2][11_2] = 6$ – matches the worked example exactly.

Python implementation

```
1 def assignBikes(workers, bikes):
2     n, m = len(workers), len(bikes)
3     INF = float('inf')
4     dp = [INF] * (1 << m)
5     dp[0] = 0
6     for i in range(n):
7         new_dp = [INF] * (1 << m)
8         wx, wy = workers[i]
9         for mask in range(1 << m):
10             if dp[mask] == INF:
11                 continue
12             for j in range(m):
13                 if mask & (1 << j):
14                     continue
15                 bx, by = bikes[j]
16                 d = abs(wx - bx) + abs(wy - by)
17                 new_mask = mask | (1 << j)
18                 new_dp[new_mask] = min(new_dp[new_mask], dp[mask] + d)
19         dp = new_dp
20     return min(dp[mask] for mask in range(1 << m)
21                if bin(mask).count('1') == n)
```

Implementation tips & tricks (the programming side)

- **A fresh `new_dp` array per worker, not in-place updates.** Because worker i 's transitions read $dp[i][\cdot]$ and write $dp[i+1][\cdot]$, updating one flat array in place would let worker i 's own writes leak into the values it still needs to read in the same pass – a classic 0/1-knapsack-style bug.
- **Skip $dp[\text{mask}] = \infty$ states early.** With $n < m$, most masks at an early i are unreachable; the `continue` prunes them for free instead of doing wasted distance computations.
- `bin(mask).count('1')` is a clear, if not the fastest, way to filter final masks by population count; for larger m prefer precomputing popcounts or Python's `int.bit_count()` (3.10+) to avoid repeated string conversion in a hot loop.

LeetCode 1595 – Minimum Cost to Connect Two Groups of Points (Hard)

Given a $\text{size1} \times \text{size2}$ cost matrix, connect every point of group 1 to **at least one** point of group 2, and every point of group 2 to **at least one** point of group 1 (a point may use *multiple* edges). Minimize the total cost of the edges used.

- Constraints (LeetCode): $\text{size1}, \text{size2} \leq 12$.
- This *looks* like Campus Bikes II's twin – but “may connect to multiple points” breaks the one-to-one structure that made Campus Bikes II a flow problem.

Worked example

cost = $[[15, 96], [36, 2]]$ (size1=size2= 2)

- Connect $g1_0 \rightarrow g2_0$ (cost 15) and $g1_1 \rightarrow g2_1$ (cost 2): every point on both sides has ≥ 1 edge.
Total = 17.
- Any other single-edge-per-point choice costs more (e.g. $g1_0 \rightarrow g2_1$ (96) is already worse than the whole answer by itself).
- **Answer:** 17.

First instinct: it looks like the same flow problem

The tempting (and *wrong*) shortcut

“This is bipartite, has costs, and needs coverage on both sides – just build the same min-cost-flow network as Campus Bikes II and run it.”

- That shortcut fails: min-cost flow (and ordinary bipartite matching) enforces a fixed *degree* at each node via edge capacities – exactly 1 connection per node in the assignment problem. Here a point may be connected to *several* points on the other side, and the goal is only “at least one,” not “exactly one.” There is no flow-conservation structure to exploit the same way.
- So what *is* the honest first instinct? Try every subset of the $\text{size1} \times \text{size2}$ possible edges, keep the cheapest subset that covers both sides.

Why this is bad

- Number of edge subsets: $2^{\text{size1} \cdot \text{size2}}$.
- At the maximum legal size, $\text{size1} = \text{size2} = 12$: that is $2^{144} \approx 2.2 \times 10^{43}$ subsets – about 10^{19} times the roughly 10^{24} grains of sand estimated to exist on Earth.
- Checking “does this subset cover both sides” is itself $O(\text{size1} \cdot \text{size2})$, making the total cost of brute force utterly hopeless even for $\text{size1}=\text{size2}=4$.

The lesson

Recognizing that a problem is *not* a clean instance of a theorem you already know (here: max-flow min-cut) is just as important a skill as recognizing when it *is* – the structural insight has to come from somewhere else.

The real structural insight

- Process group-1 points one at a time, $i = 0, \dots, \text{size1} - 1$. Track, in a bitmask over the (smaller, ≤ 12 -bit) group-2 side, **which group-2 points have already been covered** by some earlier group-1 point.
- After all of group 1 is processed, any group-2 point still uncovered *must* get an edge from *some* group-1 point – and the cheapest way to cover group-2 point j on its own is $\text{minCost2}[j] = \min_i \text{cost}[i][j]$, precomputed once.
- This gives a DP over (group-1 index, group-2 coverage mask) with only $\text{size1} \times 2^{\text{size2}}$ states – turning 2^{144} into $12 \times 4096 \approx 5 \times 10^4$.

Optimal algorithm: pseudocode

```
1: function CONNECTTWOGROUPS(cost)
2:    $s_1, s_2 \leftarrow \text{size1, size2}$ 
3:    $\text{minCost2}[j] \leftarrow \min_i \text{cost}[i][j]$  for each  $j$ 
4:   function SOLVE( $i, \text{mask}$ )
5:     if  $i = s_1$  then
6:       return  $\sum_{j \notin \text{mask}} \text{minCost2}[j]$ 
7:     end if
8:     return  $\min_j (\text{cost}[i][j] + \text{SOLVE}(i+1, \text{mask} \cup \{j\}))$ 
9:   end function
10:  return SOLVE(0,  $\emptyset$ )
11: end function
```

▷ memoized over (i, mask)

▷ cover leftover group-2 points on their own

Cost: $O(s_1 \cdot 2^{s_2} \cdot s_2)$ – at $s_1=s_2=12$, about 6×10^5 operations.

Trace on the worked example

$\text{cost} = [[15, 96], [36, 2]]$. $\text{minCost2}[0] = \min(15, 36) = 15$; $\text{minCost2}[1] = \min(96, 2) = 2$.

call	meaning	computation	value
$\text{Solve}(2, \{0, 1\})$	base, nothing left	0	0
$\text{Solve}(2, \{0\})$	base, bit 1 open	$\text{minCost2}[1]$	2
$\text{Solve}(2, \{1\})$	base, bit 0 open	$\text{minCost2}[0]$	15
$\text{Solve}(1, \{0\})$	connect $g1_1$	$\min(36+2, 2+0)$	2
$\text{Solve}(1, \emptyset)$	connect $g1_1$	$\min(36+2, 2+15)$	17
$\text{Solve}(0, \emptyset)$	connect $g1_0$	$\min(15+2, 96+17)$	17

Matches the worked example's answer of 17.

Python implementation

```
1 from functools import lru_cache
2
3 def connectTwoGroups(cost):
4     size1, size2 = len(cost), len(cost[0])
5     min_cost2 = [min(cost[i][j] for i in range(size1))
6                  for j in range(size2)]
7
8     @lru_cache(maxsize=None)
9     def solve(i, mask):
10         if i == size1:
11             return sum(min_cost2[j] for j in range(size2)
12                        if not (mask & (1 << j)))
13         return min(cost[i][j] + solve(i + 1, mask | (1 << j))
14                   for j in range(size2))
15
16     return solve(0, 0)
```

Implementation tips & tricks (the programming side)

- `@lru_cache` **needs hashable arguments**. `(i, mask)` – two integers – is hashable for free; if you were tempted to pass a set or list of covered points instead of a bitmask, memoization would silently stop working (mutable types aren't hashable), which is one more reason the bitmask encoding is the right implementation choice here, not just a space trick.
- `maxsize=None` disables the LRU eviction; with only $s_1 \times 2^{s_2} \leq 12 \times 4096$ distinct states, correctness (never recomputing, never evicting a state you still need) matters far more than bounding cache memory.
- **Precompute `min_cost2` once, outside the recursion**. Recomputing `min_i cost[i][j]` inside every base-case call would silently turn an $O(s_2)$ base case into an $O(s_1 s_2)$ one, multiplying the total cost by up to s_1 .

Summary: two lookalikes, one real lesson

- 1 **Campus Bikes II** is a genuine instance of the assignment problem – a special case of min-cost flow with unit capacities – solvable by the flow machinery this week's lecture builds toward (in practice, a tiny bitmask DP suffices at $n, m \leq 10$).
- 2 **Minimum Cost to Connect Two Groups of Points** looks identical on the surface, but “connect to possibly many” breaks the fixed-degree structure that flow relies on – it needs a different tool (coverage-bitmask DP), not a bigger flow solver.
- 3 The skill this week is building is not just “run Ford-Fulkerson” – it is recognizing *when* a problem's structure matches the Max-Flow Min-Cut Theorem and when a superficially similar problem does not.

Keep practicing

LEETCODE.md lists several more Hard flow-flavored problems for this week (*Maximum Students Taking Exam*, *Pizza With 3n Slices*, *Escape the Spreading Fire*) – for each one, ask first: is this really a flow problem, or does it only look like one?